

- Ram, PISO, SIPO, ALU

Ram

```
1 module ram #(
2     parameter ADDR_WIDTH = 8,
3     parameter DATA_WIDTH = 20
4 )
5     input wire          clk,
6     input wire          rst_n,
7     input wire          wr_en,
8     input wire [ADDR_WIDTH-1:0] addr,
9     input wire [DATA_WIDTH-1:0] din,
10    input wire          rd_en,
11    output reg [DATA_WIDTH-1:0] dout,
12    output reg          valid
13 );
14
15    reg [DATA_WIDTH-1:0] mem [0:(1<<ADDR_WIDTH)-1];
16
17    always @(posedge clk or negedge rst_n) begin
18        if (!rst_n) begin
19            dout <= {DATA_WIDTH{1'b0}};
20            valid <= 1'b0;
21        end
22        else begin
23            valid <= 1'b0;
24
25            if (wr_en)
26                mem[addr] <= din;
27
28            if (rd_en) begin
29                dout <= mem[addr];
30                valid <= 1'b1;
31            end
32        end
33    end
34
35 endmodule
```

Piso

```
piso.v
1 module piso_reg #(
2     parameter WIDTH = 20,
3     parameter ADDR_WIDTH = 8
4 )
5     input wire          clk,
6     input wire          rst_n,
7     input wire [WIDTH-1:0] parallel_in,
8     input wire          ram_valid,
9     output reg          en,
10    output reg [ADDR_WIDTH-1:0] raddr,
11    output reg          serial_out,
12    output reg          valid
13 );
14
15     reg [WIDTH-1:0] shift_reg;
16     reg [ADDR_WIDTH-1:0] address;
17     reg [5:0] count;
18     reg state;
19
20     always @(posedge clk or negedge rst_n) begin
21         if (!rst_n) begin
22             shift_reg <= {WIDTH{1'b0}};
23             address <= {ADDR_WIDTH{1'b0}};
24             raddr <= {ADDR_WIDTH{1'b0}};
25             count <= 6'd0;
26             en <= 1'b1;
27             serial_out <= 1'b0;
28             valid <= 1'b0;
29             state <= 1'b0;
30         end
31         else begin
32             if (state == 1'b0) begin
33                 en <= 1'b0;
34                 serial_out <= 1'b0;
35                 valid <= 1'b0;
36
37                 if (ram_valid) begin
38                     shift_reg <= parallel_in;
39                     state <= 1'b1;
40                 end
41             end
42             else begin
43                 en <= 1'b0;
44                 valid <= 1'b1;
45                 serial_out <= shift_reg[WIDTH-1];
46
47                 shift_reg <= shift_reg << 1;
48
49                 if (count == 6'd1) begin
50                     count <= 6'd0;
51                     valid <= 1'b0;
52                     serial_out <= 1'b0;
53                     address <= address + 1'b1;
54                     raddr <= address + 1'b1;
55                     en <= 1'b1;
56                     state <= 1'b0;
57                 end
58                 else begin
59                     count <= count - 1'b1;
60                 end
61             end
62         end
63     end
64 end
65
```

SIPO

```
1 module sipo_reg #(
2     parameter WIDTH = 20
3 );
4     input wire      clk,
5     input wire      rst_n,
6     input wire      shift_en,
7     input wire      serial_in,
8     output reg [WIDTH-1:0] parallel_out
9 );
10
11     always @(posedge clk or negedge rst_n) begin
12         if (!rst_n)
13             parallel_out <= {WIDTH{1'b0}};
14         else if (shift_en)
15             parallel_out <= {parallel_out[WIDTH-2:0], serial_in};
16         end
17     end
18 endmodule
```

ALU

```
1 module alu #(
2     parameter WIDTH = 8
3 );
4     input wire [WIDTH-1:0] in_a,
5     input wire [WIDTH-1:0] in_b,
6     input wire [2:0] opcode,
7     input wire      alu_en,
8     output reg [WIDTH-1:0] alu_out,
9     output wire      a_is_zero
10 );
11
12     assign a_is_zero = (in_a == {WIDTH{1'b0}});
13
14     always @(*) begin
15         if (!alu_en) begin
16             alu_out = {WIDTH{1'b0}};
17         end
18         else begin
19             case (opcode)
20                 3'b000: alu_out = in_a + in_b;
21                 3'b001: alu_out = in_a - in_b;
22                 3'b010: alu_out = in_a & in_b;
23                 3'b011: alu_out = in_a ^ in_b;
24                 3'b100: alu_out = in_a | in_b;
25                 3'b101: alu_out = in_a;
26                 default: alu_out = {WIDTH{1'b0}};
27             endcase
28         end
29     end
30
31 endmodule
```

Top

```
top ram piso sipo alu.v
1  module top #(
2      parameter ADDR_WIDTH = 8,
3      parameter DATA_WIDTH = 20,
4      parameter ALU_WIDTH = 8
5  )(
6      input wire          clk,
7      input wire          rst_n,
8      input wire          wr_en,
9      input wire [ADDR_WIDTH-1:0] wr_addr,
10     input wire [DATA_WIDTH-1:0] din,
11     output wire [ALU_WIDTH-1:0] alu_out,
12     output wire          a_is_zero,
13     output wire [DATA_WIDTH-1:0] parallel_out,
14     output wire          serial_out,
15     output wire          valid
16 );
17
18     wire [DATA_WIDTH-1:0] ram_dout;
19     wire ram_valid;
20     wire ram_rd_en;
21     wire [ADDR_WIDTH-1:0] ram_raddr;
22
23     wire alu_en;
24     wire [2:0] opcode;
25     wire [ALU_WIDTH-1:0] in_a;
26     wire [ALU_WIDTH-1:0] in_b;
27
28     ram #(
29         .ADDR_WIDTH(ADDR_WIDTH),
30         .DATA_WIDTH(DATA_WIDTH)
31     ) u_ram (
32         .clk(clk),
33         .rst_n(rst_n),
34         .wr_en(wr_en),
35         .addr(wr_addr),
36         .din(din),
37         .rd_en(ram_rd_en),
38         .dout(ram_dout),
39         .valid(ram_valid)
```

```

40 );
41
42 piso_reg #(
43     .WIDTH(DATA_WIDTH),
44     .ADDR_WIDTH(ADDR_WIDTH)
45 ) u_piso (
46     .clk(clk),
47     .rst_n(rst_n),
48     .parallel_in(ram_dout),
49     .ram_valid(ram_valid),
50     .en(ram_rd_en),
51     .raddr(ram_raddr),
52     .serial_out(serial_out),
53     .valid(valid)
54 );
55
56 sipo_reg #(
57     .WIDTH(DATA_WIDTH)
58 ) u_sipo (
59     .clk(clk),
60     .rst_n(rst_n),
61     .shift_en(valid),
62     .serial_in(serial_out),
63     .parallel_out(parallel_out)
64 );
65
66 assign alu_en = parallel_out[19];
67 assign opcode = parallel_out[18:16];
68 assign in_a    = parallel_out[15:8];
69 assign in_b    = parallel_out[7:0];
70
71 alu #(
72     .WIDTH(ALU_WIDTH)
73 ) u_alu (
74     .in_a(in_a),
75     .in_b(in_b),
76     .opcode(opcode),
77     .alu_en(alu_en),
78
79     .alu_out(alu_out),
80     .a_is_zero(a_is_zero)
81 );
82 endmodule

```

TB

tbLab3.v

```
1  module tb_top;
2
3      reg clk;
4      reg rst_n;
5      reg wr_en;
6      reg [7:0] wr_addr;
7      reg [19:0] din;
8
9      wire [7:0] alu_out;
10     wire a_is_zero;
11     wire [19:0] parallel_out;
12     wire serial_out;
13     wire valid;
14
15     top dut (
16         .clk(clk),
17         .rst_n(rst_n),
18         .wr_en(wr_en),
19         .wr_addr(wr_addr),
20         .din(din),
21         .alu_out(alu_out),
22         .a_is_zero(a_is_zero),
23         .parallel_out(parallel_out),
24         .serial_out(serial_out),
25         .valid(valid)
26     );
27
28     always #5 clk = ~clk;
29
30     task write_ram;
31         input [7:0] address;
32         input [19:0] data;
33         begin
34             @(negedge clk);
35             wr_en = 1'b1;
36             wr_addr = address;
37             din = data;
38             @(negedge clk);
39             wr_en = 1'b0;
40         end
41     endtask
42
43     initial begin
44         clk = 1'b0;
45         rst_n = 1'b0;
46         wr_en = 1'b0;
47         wr_addr = 8'd0;
48         din = 20'd0;
49
50         #20;
51         rst_n = 1'b1;
52
53         write_ram(8'd0, 20'b1_000_00001010_00000101);
54         write_ram(8'd1, 20'b1_001_00010100_00000101);
55         write_ram(8'd2, 20'b1_010_00001111_00000101);
56         write_ram(8'd3, 20'b1_011_00001111_00000101);
57         write_ram(8'd4, 20'b1_100_00001111_00000101);
58         write_ram(8'd5, 20'b1_101_00001111_00000101);
59         write_ram(8'd6, 20'b0_000_00001111_00000101);
60         write_ram(8'd7, 20'b1_110_00001111_00000101);
61
62         #1000;
63
64         $stop;
65     end
66
67 endmodule
```

Last Lab with Eng. Mohamed (Note: I presented the rest of them to him in the session):

Overlapping sequence analyzer (mealy)

```
overlapping Mealy.v
1  module overlapping_mealy (
2      input wire clk,
3      input wire rst_n,
4      input wire serial_in,
5      output reg detected
6  );
7
8      parameter S0 = 3'b000;
9      parameter S1 = 3'b001;
10     parameter S2 = 3'b010;
11     parameter S3 = 3'b011;
12     parameter S4 = 3'b100;
13     parameter S5 = 3'b101;
14
15     reg [2:0] state, next_state;
16
17     always @(posedge clk or negedge rst_n) begin
18         if (!rst_n)
19             state <= S0;
20         else
21             state <= next_state;
22     end
23
24     always @(*) begin
25         case (state)
26             S0: if (serial_in) next_state = S1; else next_state = S0;
27             S1: if (serial_in) next_state = S2; else next_state = S0;
28             S2: if (!serial_in) next_state = S3; else next_state = S2;
29             S3: if (serial_in) next_state = S4; else next_state = S0;
30             S4: if (!serial_in) next_state = S5; else next_state = S2;
31             S5: if (serial_in) next_state = S4; else next_state = S0;
32             default: next_state = S0;
33         endcase
34     end
35
36     always @(*) begin
37         if (state == S5 && serial_in)
38             detected = 1'b1;
39         else
40             detected = 1'b0;
41     end
42
43 endmodule
```

Overlapping sequence analyzer (moore)

```
overlapping_moore.v
1  module overlapping_moore (
2      input  wire clk,
3      input  wire rst_n,
4      input  wire serial_in,
5      output reg detected
6  );
7
8      parameter S0 = 3'b000;
9      parameter S1 = 3'b001;
10     parameter S2 = 3'b010;
11     parameter S3 = 3'b011;
12     parameter S4 = 3'b100;
13     parameter S5 = 3'b101;
14     parameter S6 = 3'b110;
15
16     reg [2:0] state, next_state;
17
18     always @(posedge clk or negedge rst_n) begin
19         if (!rst_n)
20             state <= S0;
21         else
22             state <= next_state;
23     end
24
25     always @(*) begin
26         case (state)
27             S0: if (serial_in) next_state = S1; else next_state = S0;
28             S1: if (serial_in) next_state = S2; else next_state = S0;
29             S2: if (!serial_in) next_state = S3; else next_state = S2;
30             S3: if (serial_in) next_state = S4; else next_state = S0;
31             S4: if (!serial_in) next_state = S5; else next_state = S2;
32             S5: if (serial_in) next_state = S6; else next_state = S0;
33             S6: if (serial_in) next_state = S4; else next_state = S0;
34             default: next_state = S0;
35         endcase
36     end
37
38     always @(*) begin
39         if (state == S6)
40             detected = 1'b1;
41         else
42             detected = 1'b0;
43     end
44
45 endmodule
```


Non-overlapping sequence analyzer (mealy)

```
non-overlapping_mealy.v
1  module nonoverlapping_mealy (
2      input  wire clk,
3      input  wire rst_n,
4      input  wire serial_in,
5      output reg detected
6  );
7
8      parameter S0 = 3'b000;
9      parameter S1 = 3'b001;
10     parameter S2 = 3'b010;
11     parameter S3 = 3'b011;
12     parameter S4 = 3'b100;
13     parameter S5 = 3'b101;
14
15     reg [2:0] state, next_state;
16
17     always @(posedge clk or negedge rst_n) begin
18         if (!rst_n)
19             state <= S0;
20         else
21             state <= next_state;
22     end
23
24     always @(*) begin
25         case (state)
26             S0: if (serial_in) next_state = S1; else next_state = S0;
27             S1: if (serial_in) next_state = S2; else next_state = S0;
28             S2: if (!serial_in) next_state = S3; else next_state = S2;
29             S3: if (serial_in) next_state = S4; else next_state = S0;
30             S4: if (!serial_in) next_state = S5; else next_state = S2;
31             S5: if (serial_in) next_state = S0; else next_state = S0;
32             default: next_state = S0;
33         endcase
34     end
35
36     always @(*) begin
37         if (state == S5 && serial_in)
38             detected = 1'b1;
39         else
40             detected = 1'b0;
41     end
42
43 endmodule
```

Non-overlapping sequence analyzer (moore)

```
1 module nonoverlapping_moore (  
2     input wire clk,  
3     input wire rst_n,  
4     input wire serial_in,  
5     output reg detected  
6 );  
7  
8     parameter S0 = 3'b000;  
9     parameter S1 = 3'b001;  
10    parameter S2 = 3'b010;  
11    parameter S3 = 3'b011;  
12    parameter S4 = 3'b100;  
13    parameter S5 = 3'b101;  
14    parameter S6 = 3'b110;  
15  
16    reg [2:0] state, next_state;  
17  
18    always @(posedge clk or negedge rst_n) begin  
19        if (!rst_n)  
20            state <= S0;  
21        else  
22            state <= next_state;  
23    end  
24  
25    always @(*) begin  
26        case (state)  
27            S0: if (serial_in) next_state = S1; else next_state = S0;  
28            S1: if (serial_in) next_state = S2; else next_state = S0;  
29            S2: if (!serial_in) next_state = S3; else next_state = S2;  
30            S3: if (serial_in) next_state = S4; else next_state = S0;  
31            S4: if (!serial_in) next_state = S5; else next_state = S2;  
32            S5: if (serial_in) next_state = S6; else next_state = S0;  
33            S6: next_state = S0;  
34            default: next_state = S0;  
35        endcase  
36    end  
37  
38    always @(*) begin  
39        if (state == S6)  
40            detected = 1'b1;  
41        else  
42            detected = 1'b0;  
43    end  
44  
45 endmodule
```


STA Task:

Mohamed Mossad

Path 1

$$T_{\max} = 2 + 13 = 15 \text{ ns}$$

$$T_{\min} = 2 + 1 = 3 \text{ ns}$$

path 2

$$T_{\max} = 5 + 12 = 17 \text{ ns}$$

$$T_{\min} = 5 + 2 = 7 \text{ ns}$$

Setup analysis for worst case (path₂ $T_{\max}$)

$$\text{Slack} = 25 + 7 - 2 - (4 + 6 + 17 + 4) = -1$$

∴ setup validation in path 2

Hold time analysis for worst case (path₁ $T_{\min}$)

$$\text{Slack} = (4 + 2 + 3) - (7 + 3) = -1$$

∴ Hold validation in path 1

System Verilog Lab:

Q1)

```
system verilog >  odd-even array.sv
1  module array_partition;
2      int arr[] = '{9, 7, 4, 6, 2, 8, 6, 5};
3      int even_arr[];
4      int odd_arr[];
5
6      initial begin
7          even_arr = arr.find(x) with (x % 2 == 0);
8          odd_arr = arr.find(x) with (x % 2 != 0);
9          $display("Even Array:    %p", even_arr);
10         $display("Odd Array:     %p", odd_arr);
11     end
12 endmodule
```

Q2)

```
system verilog >  maximum consecutive number.sv
1  module freq_zeros_vs_ones;
2      int arr[] = '{1, 1, 0, 1, 1, 1, 1, 0, 0, 1};
3      int ones_count;
4      int zeros_count;
5      int c;
6
7      initial begin
8          foreach (arr[i]) begin
9              c = (i == 0 || arr[i] != arr[i-1]) ? 1 : c + 1;
10
11              if (arr[i] == 1 && c > ones_count) ones_count = c;
12              if (arr[i] == 0 && c > zeros_count) zeros_count = c;
13          end
14
15          if (ones_count > zeros_count)
16              $display("1 --> %0d", ones_count);
17          else if (zeros_count > ones_count)
18              $display("0 --> %0d", zeros_count);
19          else
20              $display("0s and 1s are equal");
21      end
22 endmodule
```

Q3)

```
system verilog >  second maximum numbers.sv
1  module second_max_array_method;
2      int arr[] = '{45, 34, 67, 89, 78};
3
4      int max_val[$];
5      int filtered_q[$];
6      int sec_max[$];
7
8      initial begin
9          max_val = arr.max();
10         filtered_q = arr.find(x) with (x < max_val[0]);
11         sec_max = filtered_q.max();
12         $display("Second Max Number: %0d", sec_max[0]);
13     end
14 endmodule
```

Q4)

```
system verilog - frequency counter
1 module freq_counter_array_methods;
2   int arr[] = '{8, 3, 3, 4, 5, 6, 3, 5, 4, 6, 8, 7, 6, 4, 3, 5, 6};
3   int unique_vals[$];
4
5   initial begin
6     unique_vals = arr.unique();
7     unique_vals.sort();
8
9     foreach (unique_vals[i]) begin
10
11       $display("%0d --> %0d", unique_vals[i], arr.sum(x) with (int'(x == unique_vals[i])));
12     end
13   end
14 endmodule
```

Thank you Eng. Omar for your effort I really benifited a lot.